



**Gobierno Bolivariano
de Venezuela**

Ministerio del Poder Popular
para la Educación Universitaria

Universidad Nacional Experimental
para las Telecomunicaciones e Informática (UNETI)



República Bolivariana de Venezuela

Ministerio del Poder Popular Para la Educación Universitaria

Universidad Nacional Experimental de las Telecomunicaciones e Informática

PNF: Ingeniería en Informática

Trayecto: 3 Sección: 7A

Materia: Programación III (M2)

Ejercicios 1.2

Docente:

Ing. Carlos Márquez

Estudiantes:

Juan Echenique

V- 30.829.758

Caracas, 22 de junio de 2026



INTRODUCCIÓN

En el presente documento se expone el proceso de elaboración del ejercicios sobre el lenguaje de programación TypeScript, y Node.js como pilar principal el framework Ionic con el objetivo de facilitar su comprensión y destacar sus principales características, brindado al desarrollador las practicas mínimas necesarias en estas tecnologías y aplicaciones dentro del desarrollo web moderno.



Documentación Paso a Paso: Aplicación de IONIC

Desarrollado por: Juan Luis Echenique Gonzalez

Este documento detalla el funcionamiento, estructura y configuración del proyecto "probando ionic".

1. Descripción del Proyecto

Esta es una aplicación construida con **Node.js**, **Ionic**, **Angular** que permite a los usuarios:

Visualizar un menú de tres opciones, Home, Perfil, Contacto

Tecnologías Utilizadas

- Backend: Node.js + Express
- Framework: Ionic
- Framework: Angular
- Lenguaje: TypeScript (Tipado estático)
- Vistas: EJS (Motor de plantillas HTML)
- Estilos: CSS (Archivos estáticos en `public`)

2. Requisitos Previos

Para ejecutar este proyecto necesitas tener instalado:

- [Node.js](https://nodejs.org/) (versión 16 o superior recomendada).

```
C:\Users\User\Documents\GitHub\Ejercicio-1-2-ProgramacionIII\Ejercicio-1-ProgramacionIII> node install
node:internal/modules/cjs/loader:1424
  throw err;
  ^

Error: Cannot find module 'C:\Users\User\Documents\GitHub\Ejercicio-1-2-ProgramacionIII\Ejercicio-1-ProgramacionIII\install'
    at Module._resolveFilename (node:internal/modules/cjs/loader:1421:15)
    at defaultResolveImpl (node:internal/modules/cjs/loader:1059:19)
    at resolveForCJSWithHooks (node:internal/modules/cjs/loader:1064:22)
    at Module._load (node:internal/modules/cjs/loader:1227:37)
    at TracingChannel.traceSync (node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:245:24)
    at Module.executeUserEntryPoint [as runMain] (node:internal/modules/run_main:154:5)
    at node:internal/main/run_main_module:33:47 {
  code: 'MODULE_NOT_FOUND',
  requireStack: []
}

Node.js v24.12.0
C:\Users\User\Documents\GitHub\Ejercicio-1-2-ProgramacionIII\Ejercicio-1-ProgramacionIII>
```

Imagen 1. Instalación de Node.js

3. Instalación y Configuración

Sigue estos pasos para preparar el entorno:

Paso 1: Instalar dependencias

Abre una terminal en la carpeta raíz del proyecto y ejecuta:

```npm install```

```
C:\Users\User\Documents\GitHub\Ejercicio-1-2-ProgramacionIII\Ejercicio-1-ProgramacionIII>npm install
up to date, audited 192 packages in 786ms

22 packages are looking for funding
 run `npm fund` for details

found 0 vulnerabilities

C:\Users\User\Documents\GitHub\Ejercicio-1-2-ProgramacionIII\Ejercicio-1-ProgramacionIII>
```

Imagen 2. Instalación de npm

### 4. Instalación e inicio de Ionic

```
C:\Users\User\OneDrive\Desktop\ionic\proyecto2>ionic start sidemenu

Pick a framework!

Please select the JavaScript framework to use for your new app. To bypass this prompt next time, supply a value for the
--type option.

? Framework: Angular
```

Imagen 2. Inicio de Ionic



## 5. Una vez creado la estructura de la app

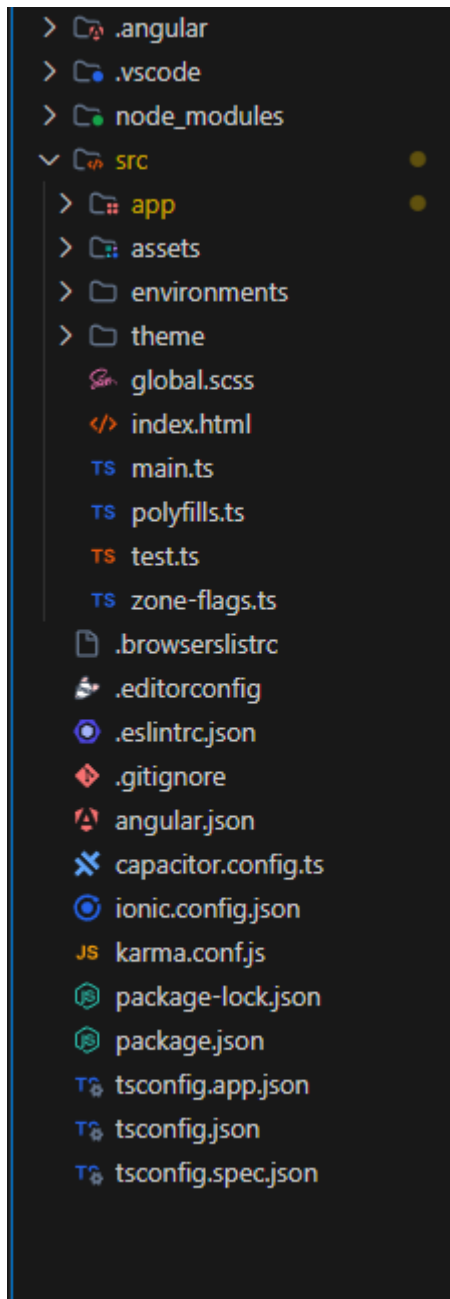


Imagen 3. Estructura

6. procedemos a crear los componentes de nuestra app

En nuestro caso crearemos 3, Home, Perfil y Contacto, estos con el comando

(\$ ionic g page "nombre del componente")

```
PS C:\Users\User\OneDrive\Desktop\ionic\proyecto2\sidemenu> ionic g page perfil
> ng.cmd generate page perfil --project=app
CREATE src/app/perfil/perfil.page.html (315 bytes)
CREATE src/app/perfil/perfil.page.spec.ts (462 bytes)
CREATE src/app/perfil/perfil.page.ts (563 bytes)
CREATE src/app/perfil/perfil.page.scss (0 bytes)
UPDATE src/app/app.routes.ts (490 bytes)
[OK] Generated page!
```

Imagen 4. Creacion de componete

7. Luego de crear los componentes comenzamos a editar y personalizar la plantilla previamente seleccionada de ionic

En este paso comenzamos insertando un nuevo tipo de color de la interfaz en la carpeta theme en el archivo (variables.scss)

```
// colores de mi app de la doctora
:root {
 --ion-color-doctora: #ec98cd;
 --ion-color-doctora-rgb: 236,152,205;
 --ion-color-doctora-contrast: #000000;
 --ion-color-doctora-contrast-rgb: 0,0,0;
 --ion-color-doctora-shade: #d086b4;
 --ion-color-doctora-tint: #eea2d2;
}

.ion-color-doctora {
 --ion-color-base: var(--ion-color-doctora);
 --ion-color-base-rgb: var(--ion-color-doctora-rgb);
 --ion-color-contrast: var(--ion-color-doctora-contrast);
 --ion-color-contrast-rgb: var(--ion-color-doctora-contrast-rgb);
 --ion-color-shade: var(--ion-color-doctora-shade);
 --ion-color-tint: var(--ion-color-doctora-tint);
}
```

Imagen 5. Inserción de código de color para tema de app

7.1 Para que los estilos se debe importar la el archivo de variables en glogal.scss

```
/* Import custom variables */
@import './theme/variables.scss';
```

Imagen 6. Importación de variable en global.css

8. Ya defino los colores nos dirigimos a componet.scss para aplicar los colores de nuestra variable doctora en los aprtado que lo necesitemos

```
16
17 ion-item.selected {
18 --color: var(--ion-color-doctora);
19 }

ion-menu.ios ion-item.selected ion-icon {
 color: var(--ion-color-doctora);
}

ion-menu.ios ion-item ion-icon {
 font-size: 24px;
 color: #4486db;
}
```

Imagen 7. Colores de iconos de la app

9. Ya los colores personalizados como queremos realizamos las modificaciones en nuestro [home.page.html](#), el cual lo mantuvimos simple con un div de bienvenida y Se insertó la imagen 1.jpg (proveniente de assets/icon/1.jpg) debajo del mensaje de bienvenida, utilizando Flexbox (display: flex; justify-content: center) para centrarla horizontalmente y se le añadieron bordes redondeados y una ligera sombra.

```
<ion-header [translucent]="true">
 <ion-toolbar color="doctora">
 <ion-buttons slot="start">
 <ion-menu-button/>
 </ion-buttons>
 <ion-title>Home</ion-title>
 </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
 <div class="ion-padding" style="background-color: var(--ion-color-doctora); color: var(--ion-color-doctora-contrast); border-radius: 8px; text-align: center; margin-top: 28px;">
 <h2>Bienvenidos al primer proyecto en Ionic de Juan Echenique</h2> Tab to Jump
 <p>es un poco sencillo espero les guste, trata de probar las rutas y estilos de este framework</p>
 </div>
 <div style="display: flex; justify-content: center; margin-top: 38px;">

 </div>
</ion-content>
```

Imagen 7. Home.page.html

10. E la parte del .ts del Home se importaron los módulos (IonButtons y IonMenuButton) desde @ionic/angular/standalone para que el botón del menú funcione correctamente.

```
c:\app> home > ts home.page.ts > % HomePage
1 import { Component, OnInit } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { FormsModule } from '@angular/forms';
4 import { IonContent, IonHeader, IonTitle, IonToolbar, IonButton, IonButtons, IonMenuButton } from '@ionic/angular/standalone';
5 import { RouterLink } from '@angular/router';
6
7 @Component({
8 selector: 'app-home',
9 templateUrl: './home.page.html',
10 styleUrls: ['./home.page.scss'],
11 standalone: true,
12 imports: [IonButton, IonButtons, IonMenuButton, IonContent, IonHeader, IonTitle, IonToolbar, CommonModule, FormsModule, RouterLink]
13 })
14 export class HomePage implements OnInit {
15
16 constructor() { }
17
18 ngOnInit() {
19 }
20
21 }
22
```

Imagen 8. Home.page.ts

11. En el código del Perfil Se construyó un formulario utilizando los componentes nativos de Ionic (<ion-list> e <ion-item>). Campos del Formulario (<ion-input>), Nombre (Tipo texto), Apellido (Tipo texto), Número de Cédula (Tipo numérico), Carrera (Tipo texto), Año Cursado (Tipo numérico)

```
1 <ion-header [translucent]="true">
2 <ion-toolbar color="doctora">
3 <ion-buttons slot="start">
4 <ion-menu-button></ion-menu-button>
5 </ion-buttons>
6 <ion-title>Perfil</ion-title>
7 </ion-toolbar>
8 </ion-header>
9
10 <ion-content class="ion-padding">
11 <ion-list>
12 <ion-item>
13 <ion-input label="Nombre" labelPlacement="floating" placeholder="Ingresa tu nombre" color="doctora"></ion-input>
14 </ion-item>
15 <ion-item>
16 <ion-input label="Apellido" labelPlacement="floating" placeholder="Ingresa tu apellido" color="doctora"></ion-input>
17 </ion-item>
18 <ion-item>
19 <ion-input label="Número de Cédula" labelPlacement="floating" type="number" placeholder="Ingresa tu cédula" color="doctora"></ion-input>
20 </ion-item>
21 <ion-item>
22 <ion-input label="Carrera" labelPlacement="floating" placeholder="Ingresa tu carrera" color="doctora"></ion-input>
23 </ion-item>
24 <ion-item>
25 <ion-input label="Año Cursado" labelPlacement="floating" type="number" placeholder="Ingresa año cursado" color="doctora"></ion-input>
26 </ion-item>
27 </ion-list>
28
29 <div class="ion-padding" style="text-align: center;">
30 <ion-button color="doctora">Guardar Información</ion-button>
31 </div>
32 </ion-content>
33
```

Imagen 9. Perfil.page.html

12. En el archivo (perfil.page.ts) se hicieron modificaciones en el controlador se añadieron a los imports los elementos del formulario (IonList, IonItem, IonInput, IonButton) así como los del menú lateral (IonButtons, IonMenuButton).

```
1 import { Component, OnInit } from '@angular/core';
2 import { CommonModule } from '@angular/common';
3 import { FormsModule } from '@angular/forms';
4 import { IonContent, IonHeader, IonTitle, IonToolbar, IonList, IonItem, IonInput, IonButton, IonButtons, IonMenuButton } from '@ionic/angular/standalone';
5
6 @Component({
7 selector: 'app-perfil',
8 templateUrl: './perfil.page.html',
9 styleUrls: ['./perfil.page.scss'],
10 standalone: true,
11 imports: [IonContent, IonHeader, IonTitle, IonToolbar, IonList, IonItem, IonInput, IonButton, IonButtons, IonMenuButton, CommonModule, FormsModule]
12 })
13 export class PerfilPage implements OnInit {
14
15 constructor() {}
16
17 ngOnInit() {}
18 }
19
20
21
```

Imagen 11 . Perfil.page.ts

13. Y por ultimo la pagina de contacto Se añadió un contenedor (div) que incluye los datos del creador: Nombre: Juan Echenique; Número de teléfono: 04241264352; Correo: [gonzalezjuanluis088@gmail.com](mailto:gonzalezjuanluis088@gmail.com) y se aplicó el mismo diseño que en la página principal, utilizando el color de fondo var(--ion-color-doctora) con texto contrastante, centrado del texto y bordes curvos (border-radius: 8px).

```
1 <ion-header [translucent]="true">
2 <ion-toolbar color="doctora">
3 <ion-buttons slot="start">
4 <ion-menu-button></ion-menu-button>
5 </ion-buttons>
6 <ion-title>Contacto</ion-title>
7 </ion-toolbar>
8 </ion-header>
9
10 <ion-content class="ion-padding">
11 <div class="ion-padding" style="background-color: var(--ion-color-doctora); color: var(--ion-color-doctora-contrast); border-radius: 8px; text-align: center; margin-top: 20px;">
12 <h2>Creador: Juan Echenique</h2>
13 <p>Número de teléfono: 04241264352</p>
14 <p>Correo: gonzalezjuanluis088@gmail.com</p>
15 </div>
16 </ion-content>
17
```

Imagen 12 . Contacto.page.html

14. Actividad funcionado

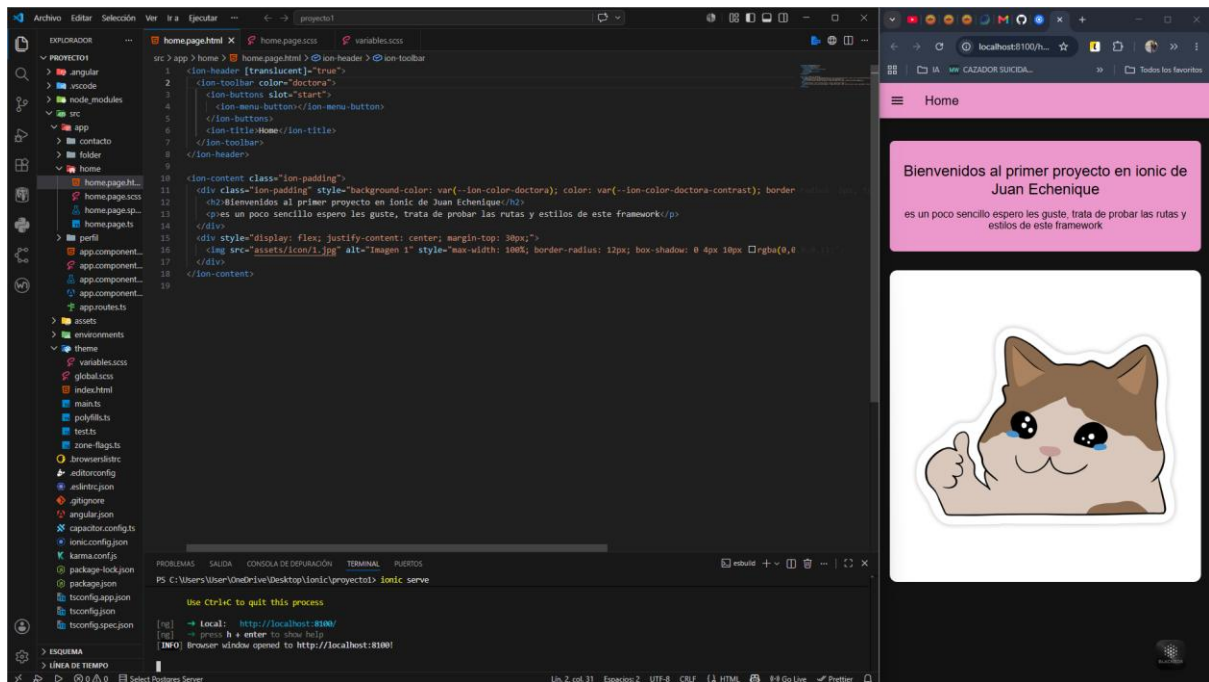


Imagen 13 . Pantalla HOME

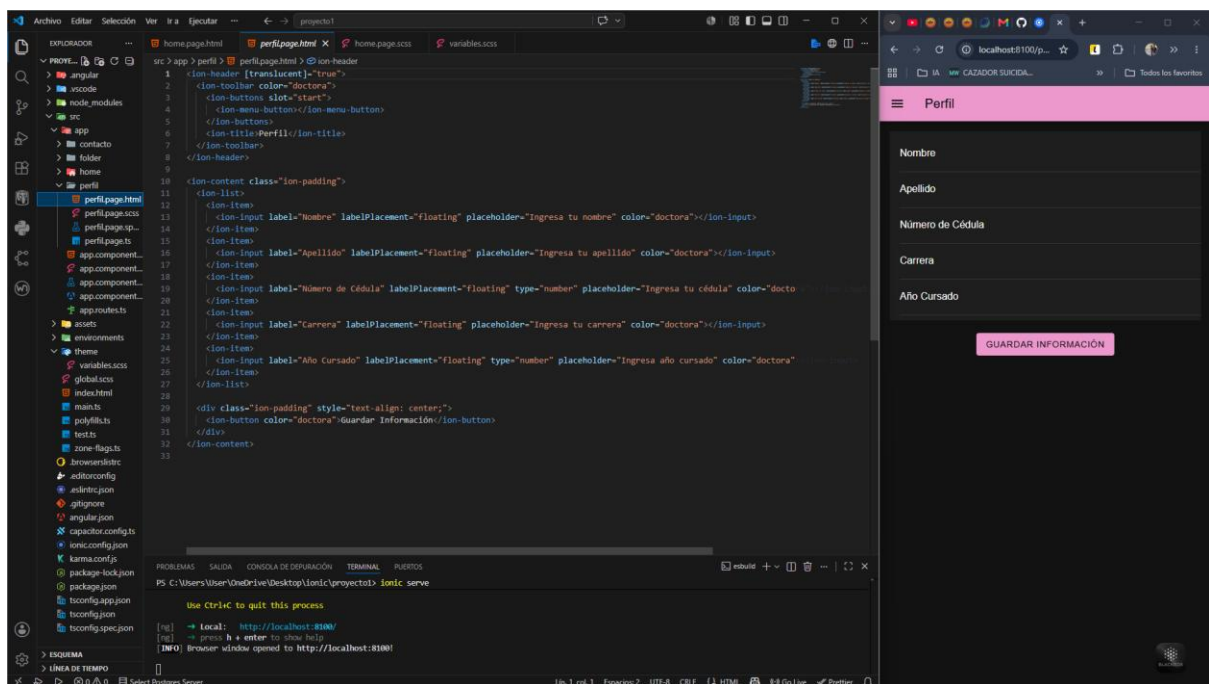


Imagen 14 . Pantalla Perfil

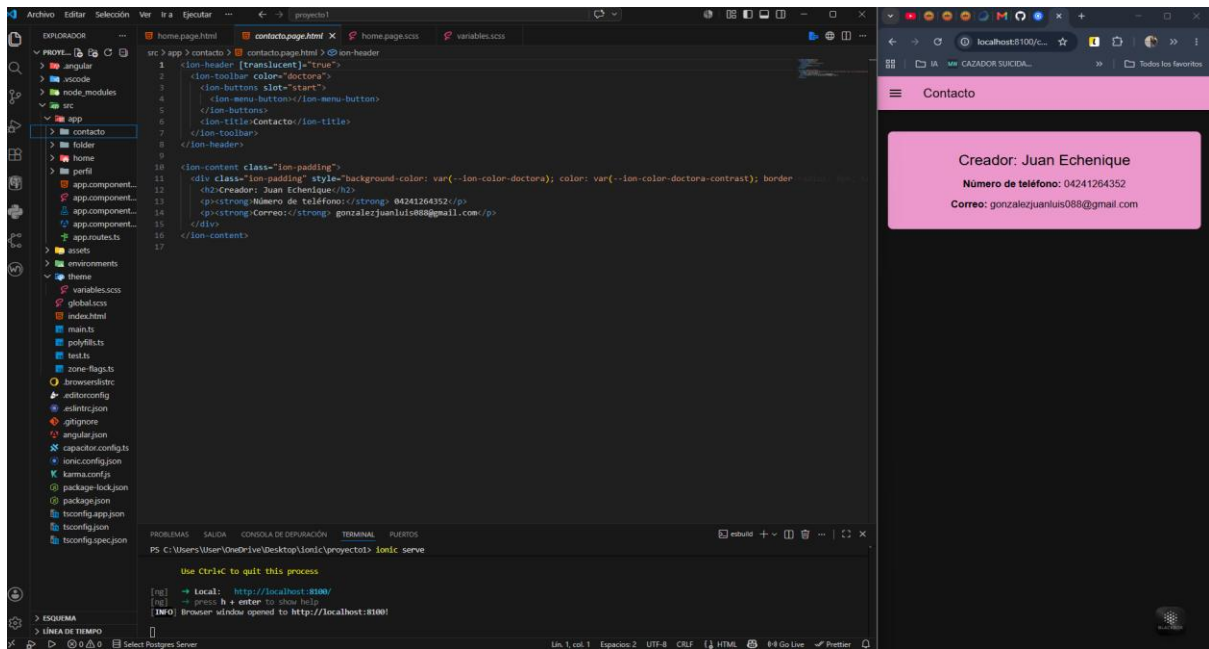


Imagen 15 . Pantalla Contacto

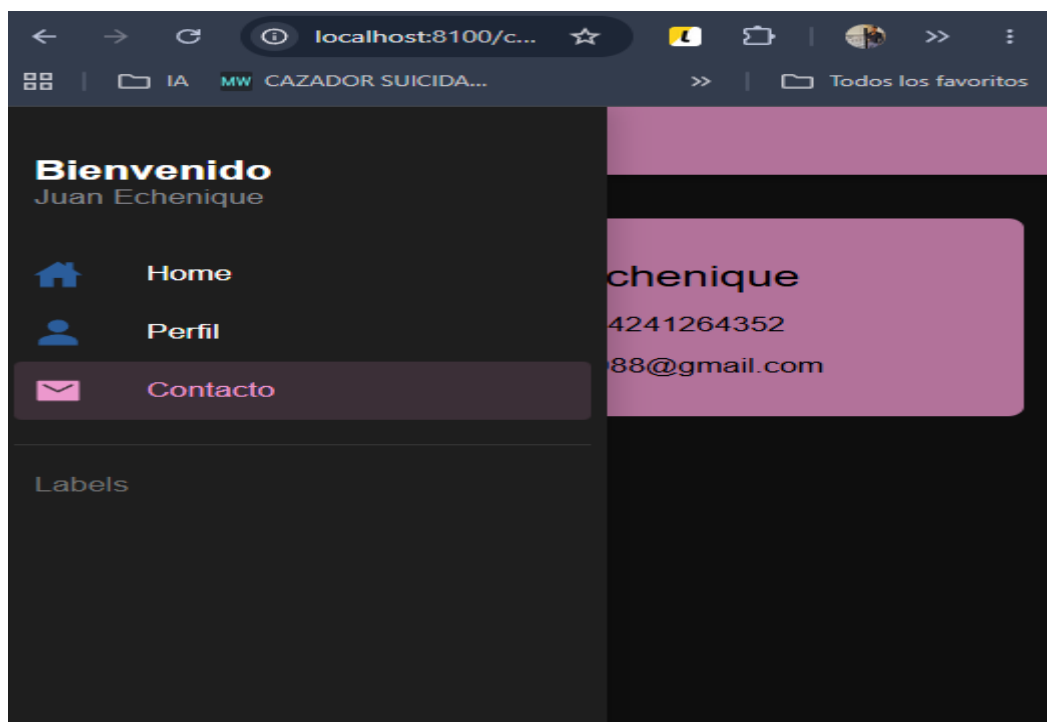


Imagen 16 . Muestra del menú

## Se actualiza la versión a 1.2

Se realizó una actualización en componente de contacto agregando un formulario donde el usuario podrá rellenar la información de dirección, su número de contacto y el número de contacto de un familiar, si tiene una emergencia. A continuación, se les presenta el código;

Se estructuraron los campos del formulario en un único objeto anidado para mantener la coherencia y facilitar la serialización al momento de enviarlo a un servicio o API:

```
67 export class ContactoPage implements OnInit {
68 contactoData = {
69 direccion: {
70 calle: '',
71 casa: '',
72 ciudad: '',
73 estado: '',
74 codigoPostal: ''
75 },
76 contacto: {
77 celular: '',
78 fijo: '',
79 correo: ''
80 },
81 familiar: {
82 nombre: '',
83 parentesco: '',
84 telefono: ''
85 }
86 };
87
88 constructor(private toastController: ToastController) {
89 addIcons([
90 homeOutline,
91 homeSharp,
92 callOutline,
93 callSharp,
94 peopleOutline, |
95 peopleSharp,
96 mailOutline,
97 mailSharp,
98 saveOutline,
99 saveSharp,
100 locationOutline,
101 locationSharp
102]);
103 }
104
105 ngOnInit() {
106 }
```

Imagen 17 . código TypeScript

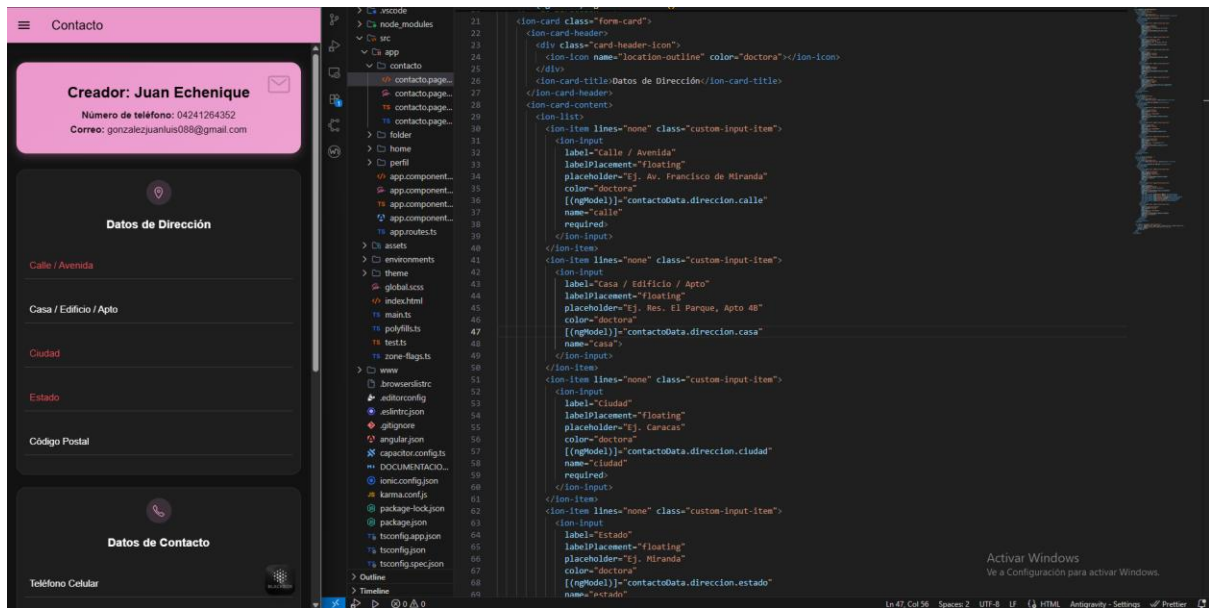


Imagen 17 .Codigo html modificado

## Repositorio del Proyecto

<https://github.com/Jun1308/Progrmacion-M2>



## CONCLUSION

El ejercicio en cuestión hizo que se aprendiera a manejar y practica con el framework ionic lo cual cumple el objetivo, aunque en primera instancias se tuvo ciertos problemas como el hecho de que no se aplicaran los estilos con la guía de la documentación oficial se pudo solventar, me parece un framework interesante aunque el hecho de estar importando propiedad por propiedad dificulta la optimización de trabajo en un entorno de producción, quizá lo que digo tenga una solución que no se encontró, pero puede que allá maneras que desconozco, en este caso la inexperiencia y la falta de tiempo para conocer la sintaxis de ionic me estaría jugar en contra. Puedo concluir que es un buen framework siempre y cuando lo comprendas de la mejor manera



## BIBLIOGRAFÍA

<https://ionicframework.com/docs/>

<https://www.youtube.com/watch?v=3oQkFwK-mVw&list=PLsngLoGbAagFEG-jwlpPhGsLzMSQ0tadP>

Derechos reservados a sus respectivos autores